# Application of Machine Learning Methods to Predict the Level of Building Damage in the Nepal Earthquake

*UE24CS352A - Machine Learning | Mini-Project*

*[VINODKUMAR L – PES1UG25AM817] | [HARSHITH K K – PES124AM370] | Section [G]*

**1. Problem Statement**

After the April 2015 Gorkha earthquake in Nepal, engineers needed to triage hundreds of thousands of buildings, but structural analysis of each building is slow. We frame this as a 3-class classification problem: given a building's structural, geographic and legal attributes, predict its damage grade (1 = low, 2 = medium, 3 = near-complete destruction). The task follows the DrivenData competition Richter's Predictor: Modeling Earthquake Damage; the metric is micro-averaged F1.

**2. Dataset**

Source: 2015 Nepal Earthquake Open Data Portal (Kathmandu Living Labs / Central Bureau of Statistics). train_values.csv: 260,601 buildings × 38 features (3 geo-level IDs, floors, age, plan area/height percentage, 8 categorical attributes such as foundation, roof and ground-floor type, 11 superstructure material flags, 11 secondary-use flags, ownership and family count). train_labels.csv: damage grade per building. test_values.csv: 86,868 unlabeled buildings (labels are held by DrivenData). The classes are imbalanced (grade 2 is the majority).

**3. Approach**

Because test labels are hidden, we hold out a stratified 20% of the labelled data for evaluation and use only the other 80% for all model selection. Three models of increasing flexibility are compared:

**K-Nearest Neighbours —** one-hot encoding, standardisation, SelectKBest(f_classif, k=20) to cut distance cost, k = 7, Euclidean distance.

**Neural network —** 2-layer (sigmoid hidden, softmax output) network implemented from scratch in NumPy with L2 regularisation and mini-batch (1000) gradient descent; hidden units = round(sqrt(n_in*n_out)); learning rate lowered until training stabilised; 3 random restarts, best chosen on an inner validation split.

**Random Forest —** ordinal-encoded features (no scaling), GridSearchCV (3-fold) over min_samples_leaf and n_estimators, Gini splits.

## 4. Implementation Overview

Python, scikit-learn, NumPy, pandas, matplotlib. src/data.py (loading and preprocessing pipelines), src/neural_net.py (from-scratch network), src/models.py (model factories), src/train.py (training, evaluation, plots, DrivenData submission file). All preprocessing lives inside scikit-learn pipelines, so no information leaks from the holdout set. One command, python run_all.py, trains everything and regenerates this report and the slides from results/metrics.json.

## 5. Results

| Model | Train F1 | Holdout F1 | Macro F1 | Fit time (s) |
|---|---|---|---|---|
| KNN | 0.6709 | 0.6461 | 0.5734 | 1 |
| Neural Network | 0.6508 | 0.6544 | 0.5666 | 99 |
| Random Forest | 0.9867 | 0.7220 | 0.6646 | 102 |

Holdout = 52,121 buildings; training = 208,480.

The source PDF also includes a train-versus-holdout F1 bar chart and confusion matrices for KNN, Neural Network, and Random Forest on page 1; these are represented by the numerical results above.

## 6. Conclusions

- Random Forest achieved the best holdout micro-F1 (0.7220); KNN was lowest (0.6461). Fastest to train: KNN (1s).
- The Random Forest shows a train-holdout gap of 0.265 (0.9867 vs 0.7220), indicating overfitting; limiting depth or raising min_samples_leaf would help.
- Per-grade recall of the best model: grade 1 = 0.49, grade 2 = 0.83, grade 3 = 0.61.
- KNN is slow at prediction time and sensitive to the feature subset; the neural network is cheap per epoch but needs learning-rate tuning; tree ensembles handle mixed categorical/numeric tabular data with minimal preprocessing.
- Future work: gradient boosting, one-hot geo-level IDs, class weighting.